

INF01151 – Sistemas Operacionais II

Aula 12 - Comunicação confiável entre processos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Problema
 - Comunicação entre processos pode falhar devido a problemas de interconexão (enlace) ou falhas no cliente ou no servidor
 - Falhas de omissão, mensagens duplicadas, mensagens entregues fora de ordem

- Métodos para tratar falhas

- Confirmações (ACK)
 - Envia uma confirmação a origem da mensagem
- Timeout
 - Tempo máximo de espera para receber confirmação, pode indicar ao cliente falha
 - **Requisição** do cliente **nunca chegou** ao servidor?
 - **Resposta** do servidor **nunca voltou** ao cliente? Operação requisitada executou ou não?

Operação idempotente?

Operação que pode ser executada repetidamente com o mesmo efeito de ter sido executada apenas uma vez

- Retransmissão

- Sempre que ocorrer um timeout reenvia a mensagem

- Descarte de mensagens duplicadas

- Se retransmissão for utilizada, servidor pode receber **várias cópias**

Servidor já enviou resposta?

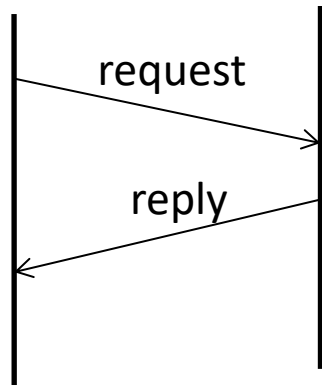
Se sim, executa operação novamente? Ou utiliza resultados da execução original?



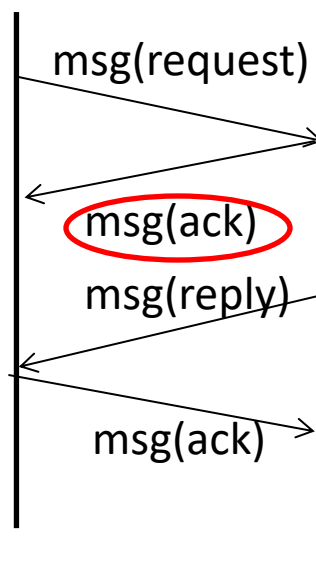
Protocolos request-reply

- Introdução ao modelo de invocação remota
 - Como processos ou objetos se comunicam em sistemas distribuídos
- Protocolos de request-reply
 - Troca de mensagens de forma bidirecional em modelos cliente/servidor
 - Fornecem suporte direto para implementação de RPC/RMI
 - Camada acima de comunicação entre processos, permitir transparência
 - Padrão de comunicação utilizado no topo de primitivas para troca de mensagens
- Normalmente, síncrono e confiável
 - Como implementar: TCP ou UDP?
 - Protocolo pode ser construído sobre UDP para evitar overhead do TCP
 - Acks, estabelecimento de conexão, controle de fluxo
 - Se UDP for usado, garantias de entrega devem ser providas pelo protocolo de request-reply

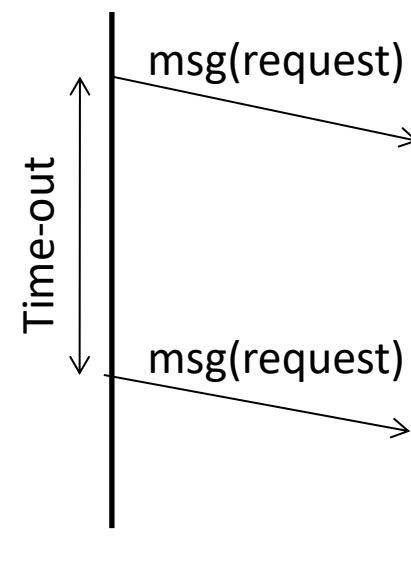
Fornecendo confiabilidade



Abstração de mais alto nível



Implementação com confirmações (explícitas)



Mensagens não confirmadas são retransmitidas (request ou reply)

Problema:

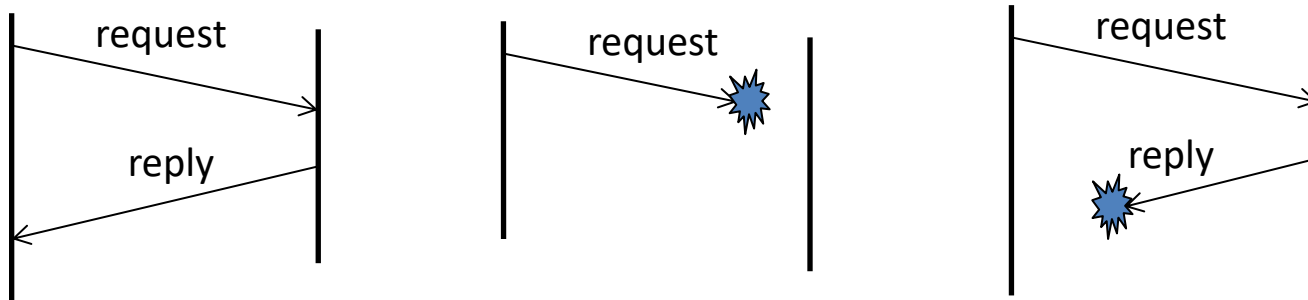
O que fazer se ack não chegar? A mensagem de *request* foi recebida? Foi a mensagem de *request* ou a mensagem de confirmação (*ack*) que deu problema?

Questão de semântica

- A semântica da comunicação depende de uma combinação de confirmações e retransmissões
 - Podem ser implementadas usando protocolos de request-reply
 - Opções de mecanismos para suportar diferentes garantias de entrega
 - Retransmissão de requisições?
 - Filtragem de mensagens duplicadas?
 - Retransmissão de resultados passados?
- Três semânticas possíveis baseadas em combinações de opções:
 - ***at most once*** (no máximo uma vez) → Modelo CORBA
 - ***at least once*** (pelo menos uma vez) → Modelo RPC
 - ***exactly once*** (exatamente uma vez) → Modelo Java RMI

Semântica no máximo uma vez (at most once)

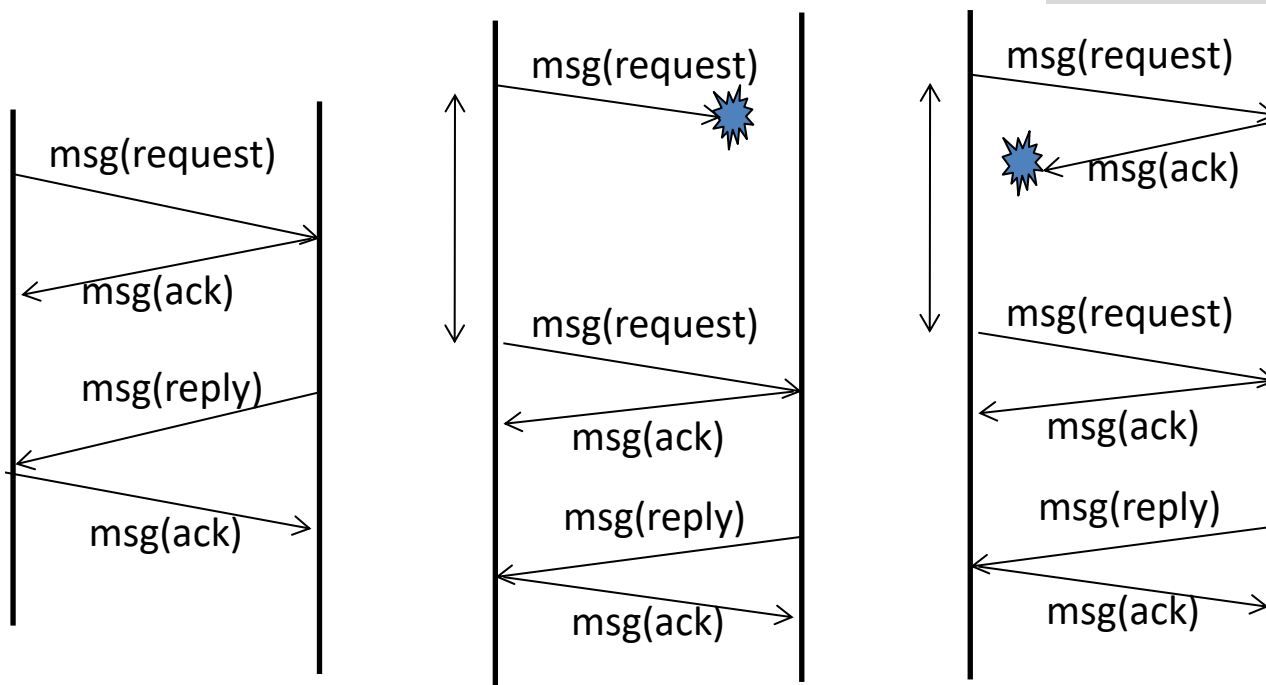
- O processo destino recebe nenhuma ou uma mensagem
 - Emissor não sabe se procedimento remoto executou ou não
- Não há nenhum tipo de confirmação ou retransmissão de mensagem
 - Método simples
 - Eficiência na comunicação já que não há *send* de confirmação ou retransmissão



Semântica pelo menos uma vez (at least once)

- Obtido com o uso de confirmações (ack) e retransmissões
 - Pode gerar mensagens duplicadas
- É garantido que o processo destino recebeu uma mensagem
 - Mas ele pode ter recebido mais de uma

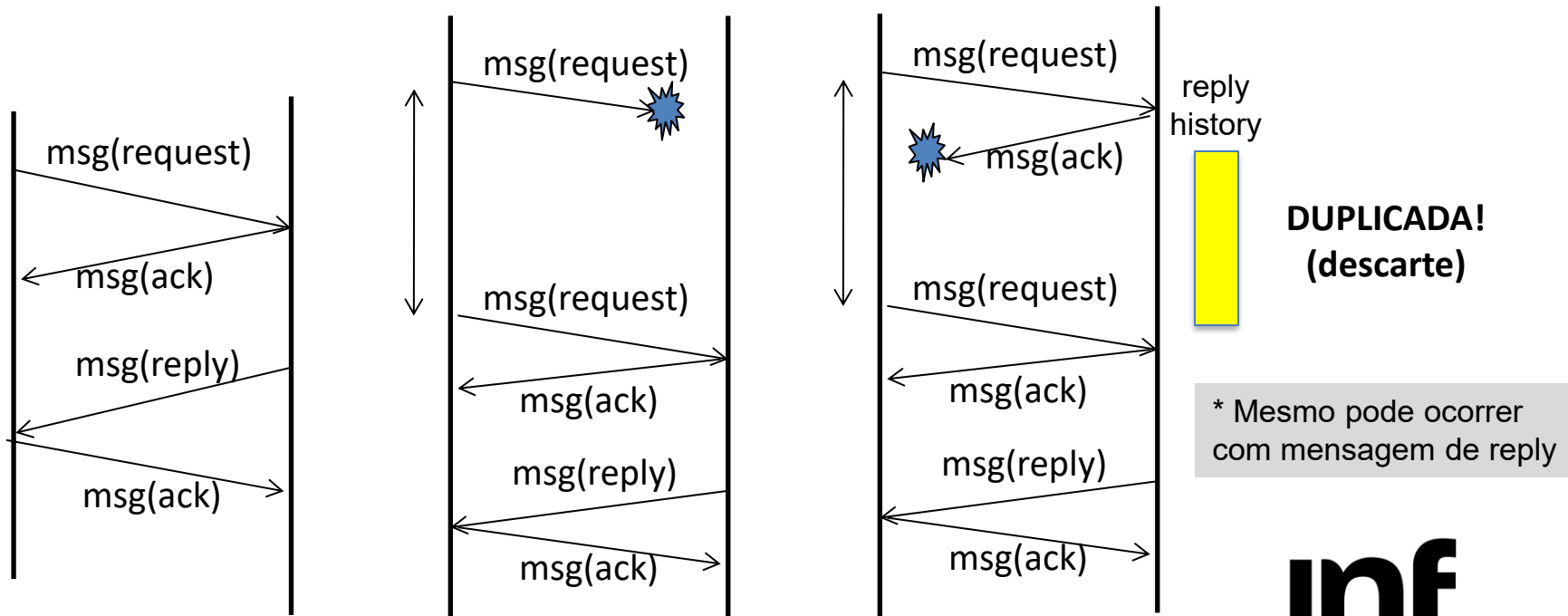
→ Quando isso não é um problema?



* Mesmo pode ocorrer com mensagem de reply

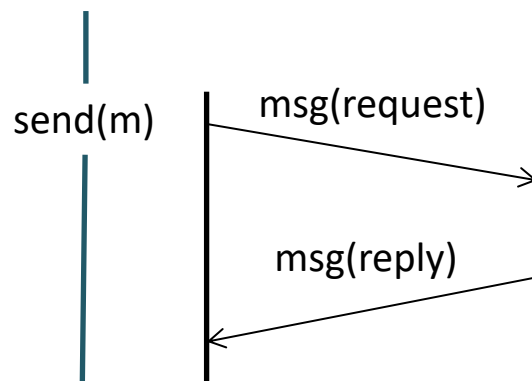
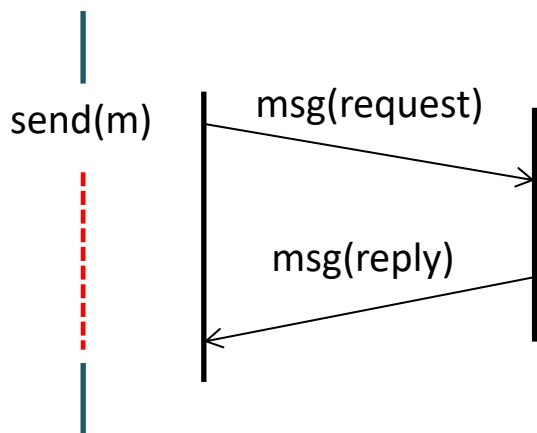
Semântica exatamente uma vez (exactly once)

- Obtida com o uso de confirmações (ack) e retransmissões
 - Mas protocolo deve reconhecer mensagens duplicadas e descartá-las
- Processo destino processa cada mensagem exatamente uma vez
 - Controle de ID, # sequência de mensagens, buferização de replies



Protocolo bloqueante e não bloqueante

- Protocolos síncronos (bloqueante) ou assíncronos (não bloqueante)
 - Diferença é se o processo emissor da mensagem de request é bloqueado até que ele receba um *reply* desta mensagem (não o ACK!!)
 - Envolvem questões de buferização do sistema
 - **Síncrono**: retorna somente após a recepção do *reply*
 - **Assíncrono**: retorna logo que o *request* for copiada para buffer do canal de comunicação



Estilos de protocolos [Spector, 1982]

- Produzem diferentes comportamentos na presença de falhas de comunicação
 - Request (R) protocol
 - Não há valor de retorno, cliente não precisa de confirmação, não bloqueia, implementado sobre UDP
 - Request-Reply (RR) protocol
 - Sem necessidades de acks pois resposta do servidor serve como confirmação
 - Request subsequente do cliente pode ser tratada como confirmação do reply
 - Pode **mascarar falhas** do UDP através de retransmissão, filtragem de duplicadas, histórico
 - Request-Reply-Ack (RRA) protocol
 - ID da requisição que vem no ack permite que o servidor descarte entradas do histórico

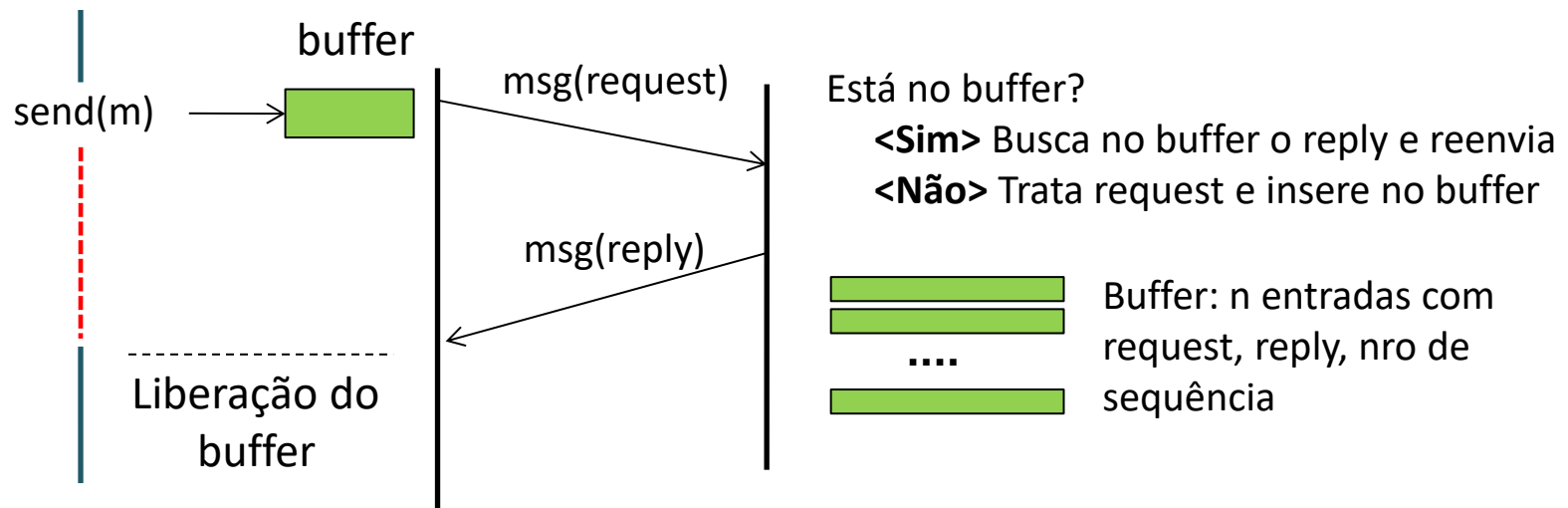
Protocolo	Cliente	Servidor	Cliente
R	Request		
RR	Request	Reply	
RRA	Request	Reply	Ack Reply

Protocolo Request-Reply (RR)

- Funcionamento:
 - Recepção do *reply* implica que destino recebeu o *request*
 - Sem confirmação explícita para o request
- Detalhes:
 - Implementa uma semântica ***at least once*** porque as mensagens de *request* podem ser duplicadas
 - Retransmissão de mensagens
 - Possível fazer uma semântica ***exactly once*** se descartar as duplicadas
 - Numerar sequencialmente os *requests*
 - Número de sequência é usado para identificar as duplicatas

Protocolo Request-Reply (RR)

- Versão bloqueante



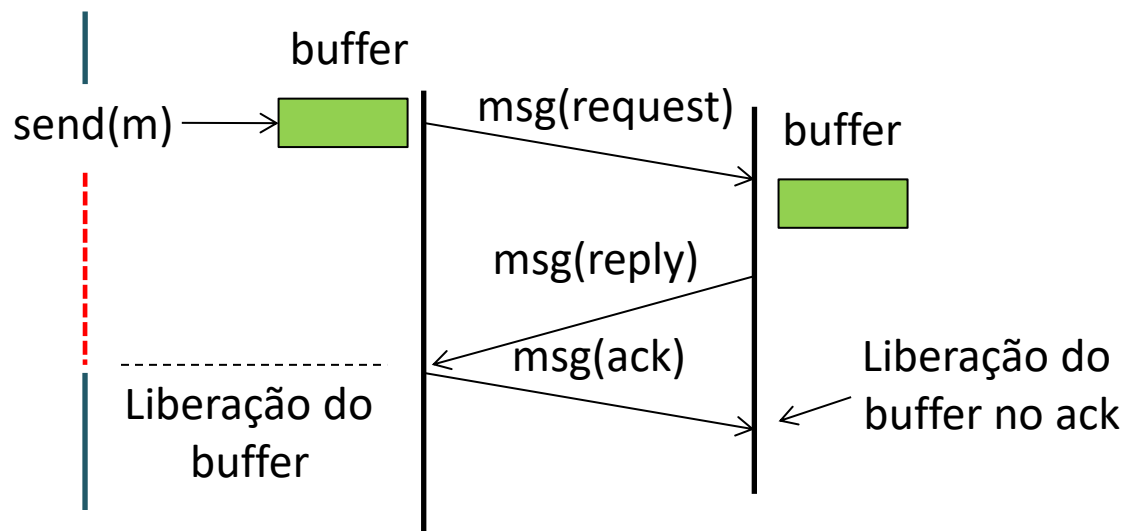
Problema?

Quando liberar as informações do buffer?
Noção de sistema *stateless* e *statefull*...

Protocolo Request-Reply-Ack (RRA)

- Funcionamento:
 - Recepção do *reply* implica que destino recebeu o *request*
 - Sem confirmação explícita para o request
 - A mensagem de reply **possui um ack explícito**
- Detalhes:
 - Implementa uma semântica **at least once** porque as mensagens de *request* podem ser duplicadas
 - Retransmissão de mensagens
 - Possível fazer uma semântica **exactly once** se descartar as duplicadas
 - Numerar sequencialmente os *requests*
 - Número de sequência é usado para identificar as duplicatas
- Quase igual ao RR, porém...

Protocolo Request-Reply-Ack (RRA)



- Implementação
 - Buffers são mantidos para o caso de necessitar retransmissão das mensagens
 - Se necessário ***exactly once***, necessário manter também os números de sequência

Características da implementação de request-reply sobre...

UDP

- Mensagens com tamanho limitado
 - Pode necessitar o uso de múltiplos datagramas para realizar um request
- Mensagens entregues de forma não confiável
 - Protocolo de request-reply precisa implementar as garantias
 - Implementação é mais difícil
- Menos overhead, se algumas funcionalidades TCP não são necessárias
 - Mensagens de tamanho fixo?
 - Todas as operações são idempotentes?
 - Perda de mensagens é tolerada?

TCP

- Mensagens sem limite de tamanho
 - Evita protocolos multipacket
- Mensagens entregues de forma confiável
 - Protocolo de request-reply não precisa lidar com retransmissões, filtragem, históricos
 - Implementação de request-reply é simples
- Protocolo TCP implica em overhead de comunicação

Slide extra: diferentes interpretações

- Semântica de invocação remota depende do modelo de falhas sendo considerado
 - Ocasional perda de mensagens, duplicações, mensagens fora de ordem

Como visto em aula*

- No máximo uma vez (*at-most-once*)
- Pelo menos uma vez (*at-least-once*)
- Exatamente uma vez (*exactly-once*)

* Não considerando falha permanente da rede ou do servidor

Coulouris⁺

- Talvez (*maybe*)
- Pelo menos uma vez (*at-least-once*)
- No máximo uma vez (*at-most-once*)

⁺ Considerando falha permanente da rede ou do servidor

- Claramente, se servidor falhar, a semântica *exactly-once* não será satisfeita...
 - **Coulouris**: considera a semântica *exactly-once* apenas para procedimentos locais

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 5 (seções 5.1, 5.2, 5.3)